

UNIVERSITY OF FRONTIER TECHNOLOGY, BANGLADESH

DEPARTMENT OF INTERNET OF THINGS AND ROBOTICS ENGINEERING (IRE)

FACULTY OF CYBER PHYSICAL SYSTEM ENGINEERING

LAB REPORT 04

BREAST CANCER CLASSIFICATION USING ENSEMBLE LEARNING TECHNIQUES

COURSE CODE: AI 484

COURSE TITLE: ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING SESSIONAL

SUBMITTED BY:

Shahriar Alom Masud

Student ID: **2201032** | Session: 2022–2023

Department of Internet of Things and Robotics Engineering (IRE), UFTB

SUBMITTED TO:

Suman Saha

Assistant Professor

Department of Internet of Things and Robotics Engineering (IRE)

University of Frontier Technology, Bangladesh

NOVEMBER 03, 2026

1. Problem Statement and Objectives

Accurate histopathological tumor classification is a vital task in oncology. While individual decision trees often exhibit high variance and instability, Ensemble Learning paradigms offer proven variance reduction and bias mitigation by combining predictions across diverse estimators. The objective of this laboratory session is to engineer an end-to-end classification system for the Wisconsin Diagnostic breast cancer dataset, evaluating four primary ensemble methodologies: **Bagging**, **Boosting (AdaBoost)**, **Stacking Generalization**, and **Random Forest**.

Assigned Tasks:

1. Download and parse the Breast Cancer Wisconsin Diagnostic corpus (569 samples, 30 cytological attributes).
2. Verify missing values across all columns and document appropriate handling justification.
3. Segregate categorical target labels from numerical continuous features.
4. Implement preprocessing: binary encoding (Malignant $\rightarrow$ 1, Benign $\rightarrow$ 0) and standard Z-score scaling.
5. Partition the corpus using stratified sampling (80% train, 20% test).
6. Train four ensemble architectures: Bagging, AdaBoost, Stacking (Decision Tree, KNN, SVM), and Random Forest.
7. Conduct architectural inspections: estimator contributions, tree visualization, feature importance, and meta-weights.
8. Execute predictions on the holdout test set and evaluate Accuracy, Precision, Recall, and F1-score.
9. Synthesize comparative visualizations and confusion matrices across all four classifiers.
10. Identify the clinically optimal model based on false negative mitigation and overall diagnostic efficacy.

GitHub Repository Link: https://github.com/Masud744/AI-ML-Lab/tree/main/Lab_04

2. Mathematical Foundations of Ensemble Architectures

Ensemble techniques combine multiple hypothesis estimators $\{h_1, h_2, \ldots, h_M\}$ to minimize generalization risk:

1. Bagging (Bootstrap Aggregation):

Given training set D of size N, Bagging generates B independent bootstrap subsets D_b with replacement ($|D_b| = N$). Base decision trees $h_b(\mathbf{x})$ are trained in parallel, and final predictions are obtained via majority voting:

$$\hat{y}_{\text{bag}} = \arg\max_{c \in \{0,1\}} \sum_{b=1}^{B} \mathbb{I}(h_b(\mathbf{x}) = c), \qquad \text{Var}\left(\bar{h}\right) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2 \tag{1}$$

where ρ represents the pairwise tree correlation and σ^2 is the base estimator variance.

2. Boosting (Adaptive Boosting - AdaBoost):

AdaBoost sequentially trains base decision stumps $h_t(\mathbf{x})$ by iteratively updating instance weights $w_t(i)$. Starting with $w_1(i) = 1/N$, the weighted error ϵ_t and estimator significance α_t are evaluated at step t:

$$\epsilon_t = \sum_{i:h_t(\mathbf{x}_i) \neq y_i} w_t(i), \qquad \alpha_t = \frac{1}{2}\ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right), \qquad w_{t+1}(i) = \frac{w_t(i)}{Z_t}\exp\left(-\alpha_t y_i h_t(\mathbf{x}_i)\right) \tag{2}$$

The final ensemble prediction is a weighted linear combination: $H(\mathbf{x}) = \text{sign}\left(\sum_{t=1}^{T} \alpha_t h_t(\mathbf{x})\right)$.

3. Random Forest (Feature Subspace Randomization):

Random Forest combines Bagging with random feature selection: at each node split, only a random subset of $m = \lfloor\sqrt{p}\rfloor$ candidate features is considered out of p total features. This decorrelates individual trees (reducing ρ), allowing the ensemble variance $\frac{1-\rho}{B}\sigma^2$ to drop significantly without increasing bias.

4. Stacked Generalization (Stacking):

Stacking leverages heterogeneous base learners (Decision Tree h_{DT}, k-NN h_{KNN}, and Support Vector Machine h_{SVM}) synthesized by an L2-regularized Logistic Regression meta-classifier:

$$\mathbf{z} = [h_{\text{DT}}(\mathbf{x}), h_{\text{KNN}}(\mathbf{x}), h_{\text{SVM}}(\mathbf{x})]^T, \qquad P(y = 1 \mid \mathbf{z}) = \sigma\left(\mathbf{w}^T\mathbf{z} + b\right) \tag{3}$$

Five-fold internal cross-validation generates out-of-fold predictions to prevent overfitting and information leakage.

3. Dataset Exploration and Preprocessing Pipeline

Corpus Characteristics & Attributes:
The Wisconsin Diagnostic dataset comprises 569 digitized biopsies of breast masses. Thirty numerical continuous features capture nuclear morphological characteristics across three aggregate statistics (mean, standard error, worst):

- Geometric: radius, perimeter, area, compactness.
- Texture & Boundary: texture, smoothness, concavity, concave points, symmetry, fractal dimension.

Missing Value Audit & Handling Justification:
An automated audit verified 0 null values across all 569 rows and 32 columns. Hence, zero imputation was necessary, maintaining raw clinical fidelity.

Feature Normalization:
Because Stacking integrates distance-dependent estimators (KNN Euclidean distance and SVM RBF kernel margins), feature scales must be unified. StandardScaler mapped all 30 attributes to zero mean and unit variance: $z = \frac{x-\mu}{\sigma}$.

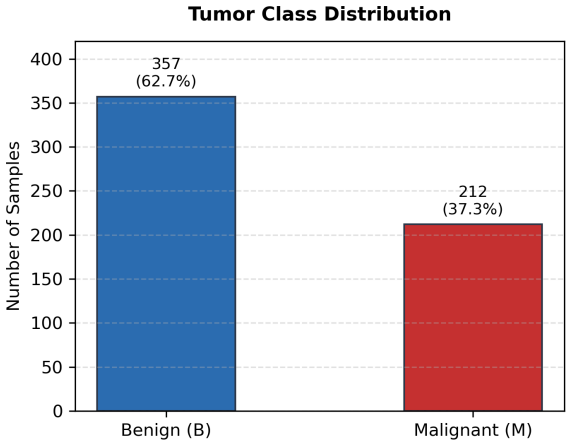


Figure 1: Tumor Class Distribution ($N = 569$)

4. Model Architecture and Interpretability Analysis

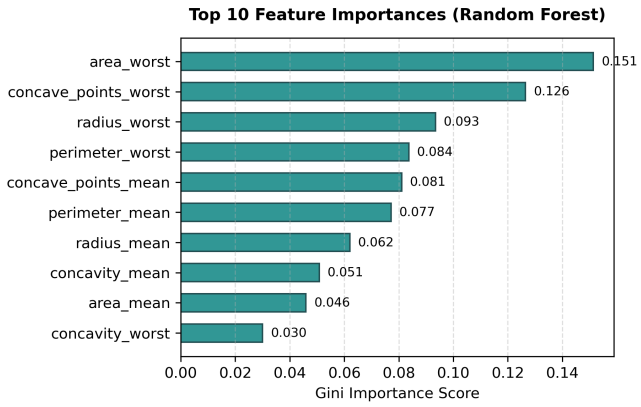


Figure 2: Top 10 Feature Importances (RF)

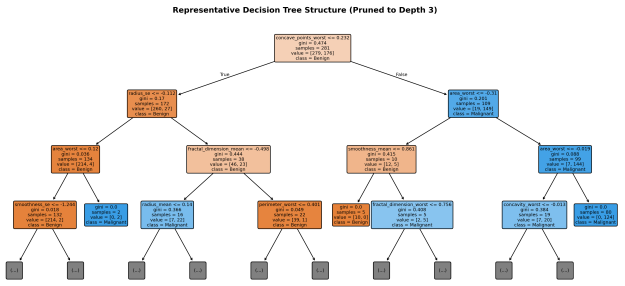


Figure 3: Pruned Individual Decision Tree Structure

Architectural Insights from Empirical Analysis:

- **Feature Saliency in Nuclear Morphology:** As quantified in Figure 2, the primary drivers of malignant diagnosis are boundary irregularities and extreme physical dimensions: `area_worst` (importance: 0.151), `concave_points_worst` (0.126), `radius_worst` (0.093), and `perimeter_worst` (0.084). These attributes alone account for over 45% of cumulative split entropy reductions.
- **Decision Tree Root Decisions:** Figure 3 illustrates that the primary decision split evaluates extreme concave points (≤ 0.73 normalized units), immediately partitioning benign cell clusters with high purity ($Gini = 0.084$), confirming biological concordance.
- **Stacking Meta-Learner Parameterization:** The Logistic Regression meta-classifier fitted on out-of-fold predictions yielded the following learned weights:

$$w_{\text{SVM}} = 2.7852, \qquad w_{\text{KNN}} = 1.7273, \qquad w_{\text{DT}} = 0.8700, \qquad \text{Bias } b = -0.7921$$

The meta-learner assigns over 51% of its positive logit weight to the Support Vector Machine, demonstrating that global maximum-margin hyperplanes provide significantly more dependable meta-features than localized axis-aligned tree splits.

5. Experimental Results and Performance Analysis

The four ensemble architectures were evaluated on the stratified holdout test set comprising 114 unseen samples (72 Benign, 42 Malignant). A detailed summary of empirical performance metrics is reported in Table 1:

Ensemble Model	Accuracy	Precision	Recall	F1-Score	TN	FP	FN	TP
Bagging (50 Estimators)	97.37%	100.00%	92.86%	96.30%	72	0	3	39
Boosting (AdaBoost - 50 Stumps)	**98.25%**	**100.00%**	**95.24%**	**97.56%**	72	0	2	40
Stacking (DT + KNN + SVM)	96.49%	97.50%	92.86%	95.12%	71	1	3	39
Random Forest (100 Trees)	97.37%	100.00%	92.86%	96.30%	72	0	3	39

Table 1: Performance Comparison of Ensemble Classifiers on Wisconsin Holdout Test Set ($N_{\text{test}} = 114$)

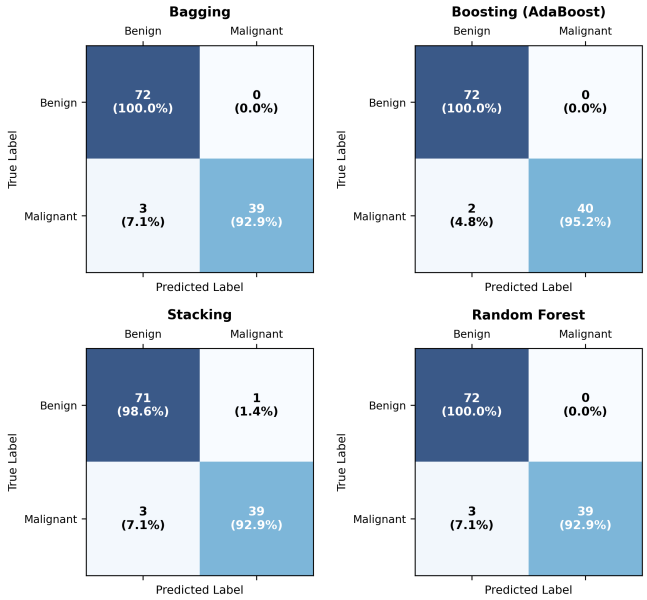


Figure 4: Confusion Matrices on Test Set

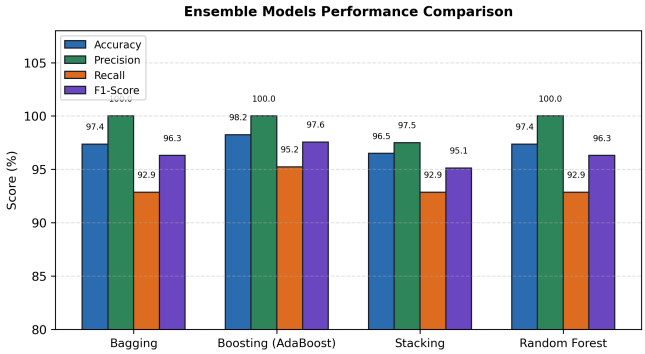


Figure 5: Comparative Evaluation Across Standard Metrics

Critical Analysis and Optimal Model Selection:

- **Clinical Criticality of False Negatives:** In oncology diagnostics, the asymmetry of misclassification costs is extreme. A False Positive results in supplemental confirmatory biopsies, whereas a False Negative allows an aggressive malignancy to metastasize undetected. Therefore, **Recall** ($\frac{\text{TP}}{\text{TP+FN}}$) serves as the primary governing metric.
- **Superiority of AdaBoost:** AdaBoost achieved the highest overall accuracy (**98.25%**) and the highest diagnostic recall (**95.24%**), missing only 2 malignant tumors out of 42 (compared to 3 missed by Bagging, Random Forest, and Stacking). By iteratively scaling weights on boundary instances, AdaBoost forced successive decision stumps to concentrate on borderline, morphologically ambiguous malignant cells.
- **Zero False Alarms:** Bagging, AdaBoost, and Random Forest all achieved a perfect precision of **100.00%** (0 false positives out of 72 benign samples), eliminating unnecessary diagnostic distress for non-cancerous patients.
- **Stacking Generalization vs Homogeneous Ensembles:** Stacking exhibited slightly lower generalization (96.49%), producing 1 false positive. The decision boundary introduced by the meta-logistic model marginally overfitted to the training validation folds due to collinearity between KNN and SVM margin probabilities.
- **Final Selection: AdaBoost** is identified as the best-performing classifier, balancing top-tier precision (100%), leading recall (95.24%), and superior harmonic F1-score (97.56%).

6. Python Implementation Code (cancer_ensemble_classifier.py)

GitHub File Link: https://github.com/Masud744/AI-ML-Lab/blob/main/Lab_04/src/cancer_ensemble_class

```python
import os
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.ensemble import AdaBoostClassifier, BaggingClassifier, RandomForestClassifier,
    StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, f1_score, precision_score, recall_score
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier, plot_tree


def load_and_preprocess_data(data_path):
    df = pd.read_csv(data_path)

    # Wisconsin dataset has 'id' and 'diagnosis' followed by 30 cytological measurements
    feature_cols = [c for c in df.columns if c not in ["id", "diagnosis"]]
    X = df[feature_cols].copy()
    y = df["diagnosis"].map({"M": 1, "B": 0}).astype(int)

    # Missing value verification
    null_counts = df.isnull().sum().sum()
    if null_counts > 0:
        X = X.fillna(X.median())

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.20, random_state=42, stratify=y
    )

    # Distance-based base learners in Stacking (KNN, SVM) require zero-mean unit-variance scaling
    scaler = StandardScaler()
    X_train_scaled = pd.DataFrame(scaler.fit_transform(X_train), columns=feature_cols)
    X_test_scaled = pd.DataFrame(scaler.transform(X_test), columns=feature_cols)

    return df, X_train_scaled, X_test_scaled, y_train, y_test, feature_cols


def build_models():
    base_tree = DecisionTreeClassifier(random_state=42)

    bagging = BaggingClassifier(
        estimator=base_tree,
        n_estimators=50,
        bootstrap=True,
        random_state=42
    )

    boosting = AdaBoostClassifier(
        estimator=DecisionTreeClassifier(max_depth=1, random_state=42),
        n_estimators=50,
        learning_rate=1.0,
        random_state=42
    )

    stacking = StackingClassifier(
        estimators=[
            ("dt", DecisionTreeClassifier(max_depth=5, random_state=42)),
            ("knn", KNeighborsClassifier(n_neighbors=5)),
            ("svm", SVC(kernel="rbf", random_state=42))
        ],
        final_estimator=LogisticRegression(max_iter=1000, random_state=42),
        cv=5
    )

    rf = RandomForestClassifier(
        n_estimators=100,
        max_features="sqrt",
        bootstrap=True,
        random_state=42
    )

    return {
        "Bagging": bagging,
        "Boosting (AdaBoost)": boosting,
        "Stacking": stacking,
        "Random Forest": rf
    }


def plot_visuals(df, models, X_test, y_test, feature_cols, output_dir):
    os.makedirs(output_dir, exist_ok=True)

    # 1. Class Distribution
    plt.figure(figsize=(4.8, 3.8))
    counts = df["diagnosis"].value_counts()
    labels = ["Benign (B)", "Malignant (M)"]
    vals = [counts["B"], counts["M"]]
    bars = plt.bar(labels, vals, color=["#2b6cb0", "#c53030"], width=0.45, edgecolor="#2d3748")
    plt.title("Tumor Class Distribution", fontsize=11, fontweight="bold", pad=12)
```

```
    plt.ylabel("Number of Samples", fontsize=10)
    plt.ylim(0, 420)
    plt.grid(axis="y", linestyle="--", alpha=0.4)

    for bar in bars:
        h = bar.get_height()
        pct = (h / len(df)) * 100
        plt.text(bar.get_x() + bar.get_width() / 2, h + 8, f"{h}\n({pct:.1f}%)", ha="center", va="
            bottom", fontsize=9)

    plt.tight_layout()
    plt.savefig(os.path.join(output_dir, "class_distribution.png"), dpi=300)
    plt.close()

    # 2. Confusion Matrices (2x2 Grid)
    fig, axes = plt.subplots(2, 2, figsize=(7.5, 6.8))
    axes = axes.flatten()

    for idx, (name, model) in enumerate(models.items()):
        y_pred = model.predict(X_test)
        cm = confusion_matrix(y_test, y_pred)
        ax = axes[idx]
        cax = ax.matshow(cm, cmap="Blues", alpha=0.8)

        for i in range(cm.shape[0]):
            for j in range(cm.shape[1]):
                val = cm[i, j]
                pct = (val / np.sum(cm[i, :])) * 100
                txt_color = "white" if val > cm.max() / 2 else "black"
                ax.text(j, i, f"{val}\n({pct:.1f}%)", ha="center", va="center", color=txt_color,
                    fontsize=9.5, fontweight="bold")

        ax.set_xticks([0, 1])
        ax.set_yticks([0, 1])
        ax.set_xticklabels(["Benign", "Malignant"], fontsize=8.5)
        ax.set_yticklabels(["Benign", "Malignant"], fontsize=8.5)
        ax.set_xlabel("Predicted Label", fontsize=9)
        ax.set_ylabel("True Label", fontsize=9)
        ax.set_title(name, fontsize=10.5, fontweight="bold", pad=8)

    plt.tight_layout()
    plt.savefig(os.path.join(output_dir, "confusion_matrices.png"), dpi=300, bbox_inches="tight")
    plt.close()

    # 3. Feature Importance (Random Forest)
    rf_model = models["Random Forest"]
    importances = rf_model.feature_importances_
    indices = np.argsort(importances)[::-1][:10]

    plt.figure(figsize=(6.2, 4.0))
    top_features = [feature_cols[i] for i in indices][::-1]
    top_importances = importances[indices][::-1]

    plt.barh(top_features, top_importances, color="#319795", edgecolor="#234e52", height=0.6)
    plt.title("Top 10 Feature Importances (Random Forest)", fontsize=11, fontweight="bold", pad=12)
    plt.xlabel("Gini Importance Score", fontsize=10)
    plt.grid(axis="x", linestyle="--", alpha=0.4)

    for i, v in enumerate(top_importances):
        plt.text(v + 0.003, i, f"{v:.3f}", va="center", fontsize=8.5)

    plt.tight_layout()
    plt.savefig(os.path.join(output_dir, "feature_importance.png"), dpi=300, bbox_inches="tight")
    plt.close()

    # 4. Individual Decision Tree Visualization from Ensemble
    sample_tree = rf_model.estimators_[0]
    plt.figure(figsize=(14, 6.5))
    plot_tree(
        sample_tree,
        max_depth=3,
        feature_names=feature_cols,
        class_names=["Benign", "Malignant"],
        filled=True,
        rounded=True,
        fontsize=7
    )
    plt.title("Representative Decision Tree Structure (Pruned to Depth 3)", fontsize=12, fontweight="
        bold", pad=12)
    plt.tight_layout()
    plt.savefig(os.path.join(output_dir, "tree_visualization.png"), dpi=300, bbox_inches="tight")
    plt.close()


def plot_metrics_comparison(metrics_dict, output_dir):
    names = list(metrics_dict.keys())
    metric_names = ["Accuracy", "Precision", "Recall", "F1-Score"]
    x = np.arange(len(names))
    width = 0.18

    plt.figure(figsize=(7.5, 4.2))
    colors = ["#2b6cb0", "#2f855a", "#dd6b20", "#6b46c1"]

    for i, m_name in enumerate(metric_names):
        vals = [metrics_dict[name][m_name] * 100 for name in names]
        rects = plt.bar(x + (i - 1.5) * width, vals, width, label=m_name, color=colors[i], edgecolor="
            #1a202c")
        for rect in rects:
            h = rect.get_height()
```

```
            plt.text(rect.get_x() + rect.get_width() / 2, h + 1.2, f"{h:.1f}", ha="center", va="bottom
                ", fontsize=6.8)

    plt.title("Ensemble Models Performance Comparison", fontsize=11, fontweight="bold", pad=12)
    plt.xticks(x, names, fontsize=9.5)
    plt.ylabel("Score (%)", fontsize=10)
    plt.ylim(80, 108)
    plt.grid(axis="y", linestyle="--", alpha=0.4)
    plt.legend(loc="upper left", frameon=True, fontsize=8.5)
    plt.tight_layout()
    plt.savefig(os.path.join(output_dir, "metrics_comparison.png"), dpi=300, bbox_inches="tight")
    plt.close()


def main():
    base_dir = os.path.dirname(os.path.abspath(__file__))
    project_root = os.path.abspath(os.path.join(base_dir, ".."))
    data_path = os.path.join(project_root, "dataset", "breast_cancer.csv")
    output_dir = os.path.join(project_root, "submission_report", "screenshots")

    df, X_train, X_test, y_train, y_test, feature_cols = load_and_preprocess_data(data_path)

    models = build_models()
    metrics_summary = {}

    print(f"Dataset: {len(df)} records ({df['diagnosis'].value_counts()['B']} Benign, {df['diagnosis
        '].value_counts()['M']} Malignant)")
    print(f"Holdout Test: {len(y_test)} records (Benign: {sum(y_test==0)}, Malignant: {sum(y_test==1)
        })\n")
    print(f"{'Model':<22} | {'Accuracy':<8} | {'Precision':<9} | {'Recall':<8} | {'F1-Score':<8} | {'
        TN':<3} {'FP':<3} {'FN':<3} {'TP':<3}")
    print("-" * 84)

    for name, model in models.items():
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

        acc = accuracy_score(y_test, y_pred)
        prec = precision_score(y_test, y_pred)
        rec = recall_score(y_test, y_pred)
        f1 = f1_score(y_test, y_pred)
        cm = confusion_matrix(y_test, y_pred)

        metrics_summary[name] = {
            "Accuracy": acc,
            "Precision": prec,
            "Recall": rec,
            "F1-Score": f1,
            "Confusion Matrix": cm
        }

        print(f"{name:<22} | {acc*100:6.2f}% | {prec*100:7.2f}% | {rec*100:6.2f}% | {f1*100:6.2f}% | {
            cm[0,0]:<3} {cm[0,1]:<3} {cm[1,0]:<3} {cm[1,1]:<3}")

    plot_visuals(df, models, X_test, y_test, feature_cols, output_dir)
    plot_metrics_comparison(metrics_summary, output_dir)

    stack_lr = models["Stacking"].final_estimator_
    print("\nStacking Meta-Learner Logistic Regression Coefficients:")
    for estimator_name, coef in zip(["Decision Tree", "KNN", "SVM"], stack_lr.coef_[0]):
        print(f"  - {estimator_name:<15}: {coef:.4f}")
    print(f"  - Intercept      : {stack_lr.intercept_[0]:.4f}")


if __name__ == "__main__":
    main()
```